



Deep Learning: Handwritten Digit Recognition

Submitted in partial fulfillment of completion of the course

AI Micro Degree

Year 2026

Submitted by:

- **Bhoomi** - Worked on coding implementation and & Video Editing for demonstration.
- **Kashish Gautam** – Worked & Prepared the Presentation, Documentation, basic coding support.
- **Ashi** – Worked on coding implementation & Assisted in data understanding and preprocessing.
- **Khushi Kashyap** – Dataset Handling & Contributed the Documentation.

Under Guidance of:

Deepika Singh (Edunet)

Abstract

Handwritten Digit Recognition is a classic problem in the field of computer vision and deep learning. This project focuses on building a neural network model to accurately recognize handwritten digits (0–9) using the MNIST dataset. The dataset consists of grayscale images of size 28×28 pixels, where each image represents a single digit.

Acknowledgement

We would like to express our sincere gratitude to our teacher and institution for their constant guidance, support, and encouragement throughout the completion of this project on Handwritten Digit Recognition. Their valuable suggestions helped us to understand the concepts better and successfully complete this work. We would also like to thank the developers of Python, TensorFlow, Keras, and Streamlit for providing powerful tools that made this project possible.

Team Composition and Workload Division

- **Bhoomi** - Worked on coding implementation and & Video Editing for demonstration.
- **Kashish Gautam** – Worked & Prepared the Presentation, Documentation, basic coding support.
- **Ashi** – Worked on coding implementation & Assisted in data understanding and preprocessing.
- **Khushi Kashyap** – Dataset Handling & Contributed the Documentation.

Table of Contents

Abstract.....	i
Acknowledgement	ii
1. Introduction to the Problem	1
2. Literature Review	2
3. Proposed Solution	3
4. Project Requirements.....	4
4.1 Technology Stack	4
4.2 Hardware Requirements	5
4.3 Software Requirements	6
4.4 Deployment Environment	7
5. User Requirements	8
6. Design Documentation	9
7. Implementation Details	10
8. Testing and Validation	11
9. Deployment	12
10.Future Scope	13
11.Conclusion	14
Appendix A – Project Code	15
Appendix B – Project Screenshots	16
Appendix C – Abbreviations	17
References	18

1. Introduction to Problem

Handwritten digit recognition is the process of identifying digits (0–9) written by hand. It is a common problem in machine learning and computer vision. Humans can easily read handwritten numbers, but for a computer, it is difficult because every person writes in a different style.

2. Literature Review

Handwritten digit recognition has been widely studied in the field of machine learning and computer vision. Earlier methods used traditional algorithms like K-Nearest Neighbors (KNN), Support Vector Machines (SVM), and basic feature extraction techniques to classify digits. These methods worked well on simple datasets but had limitations in handling complex patterns and variations in handwriting.

3. Proposed Solution

In this project, we propose a deep learning-based solution to recognize handwritten digits (0–9). We use the MNIST dataset to train a neural network model that can learn patterns from handwritten images. The labels are converted into a binary format using one-hot encoding. After pre-processing, the data is split into training and testing sets.

4. Requirements

a. Technology Stack

- Python
- NumPy
- Pandas
- matplotlib. (Pyplot)
- TensorFlow / Keras
- Scikit-learn
- Streamlit
- PIL (Python Imaging Library)

b. Hardware

- Computer/Laptop
- Minimum 8 GB RAM
- Intel i5 or equivalent processor
- Webcam or camera (optional for image input)

c. Software

- Windows / Linux Operating System
- Python IDE (Jupyter Notebook / PyCharm / VS Code)
- Required Python Libraries (OpenCV, NumPy, Pandas, Matplotlib, Scikit-learn, Streamlit, PIL (Python Imaging Library))

d. Deployment Environment

- The deployment environment for this project is a local machine using a web-based Interface. The trained model is integrated with a Streamlit application, which runs on the local server and allows users to interact with the system easily.
- The application is executed using Python and Streamlit, where the user uploads a handwritten digit image, and the model predicts the output in real time

5. User Requirements

- The user should have basic knowledge of how to use a computer and upload image files.
- The system is designed to be simple and user-friendly so that any user can easily interact with it without technical skills.
- The user must have a device with an internet browser to access the Streamlit application (if deployed online) or a system with Python installed (if running locally).

6. Design Documentation

The system follows a modular design approach.

Design Flow:

1. Image Input (MNIST dataset or user-uploaded image)
2. Image pre-processing (grayscale, resize to 28×28, normalization)
3. Data Preparation (reshaping and one-hot encoding)
4. Model Training (neural network using TensorFlow/Keras)
5. Digit Classification (prediction using softmax output layer)
6. Data Analysis & Visualization (accuracy graph and confusion matrix)

7. Implementation Details

The project is implemented using Python and the MNIST dataset in CSV format. First, the dataset is loaded using Pandas and divided into input features and output labels. The pixel values are normalized by scaling them between 0 and 1, and the images are reshaped into 28×28 format to make them suitable for the neural network. The labels are converted into one-hot encoded format for multi-class classification.

8. Testing

- The system is tested using multiple handwritten digit images from the dataset as well as user-provided images. Testing ensures:
- Correct recognition of handwritten digits (0–9).
- Accurate image pre-processing such as resizing, normalization, and grayscale conversion.
- Proper functioning of the rule-based classification system.

9. Deployment

The system is deployed using a Streamlit web application, which provides an interactive interface for users to upload handwritten digit images and get real-time predictions. The trained model is saved in a .h5 file format and loaded into the Streamlit app for inference.

10. Future Scope

In future, the system can be improved by using advanced deep learning models like Convolutional Neural Networks (CNN) to achieve higher accuracy and better performance. It can also be extended to recognize not only digits but also handwritten letters and complete words. A real-time drawing feature can be added so that users can draw digits directly on the screen for prediction.

11. Conclusion

The Handwritten Digit Recognition project successfully demonstrates the use of deep learning for image classification. The model is trained using the MNIST dataset and is able to accurately recognize digits from 0 to 9. By applying proper pre-processing techniques and a neural network model, the system achieves good performance and reliable predictions.

Appendix A Project Code

1. Train model

```
• # Import libraries
• import pandas as pd
• import numpy as np
• import tensorflow as tf
• from tensorflow.keras import layers, models
•
• # Load dataset
• train = pd.read_csv("data/mnist_train.csv")
• test = pd.read_csv("data/mnist_test.csv")
•
• # Split features and labels
• x_train = train.iloc[:, 1:].values
• y_train = train.iloc[:, 0].values
•
• x_test = test.iloc[:, 1:].values
• y_test = test.iloc[:, 0].values
•
• # Normalize pixel values
• x_train = x_train / 255.0
• x_test = x_test / 255.0
•
• # Reshape into 28x28 images
• x_train = x_train.reshape(-1, 28, 28, 1)
• x_test = x_test.reshape(-1, 28, 28, 1)
•
• # Build CNN model
• model = models.Sequential([
•     layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
•     layers.MaxPooling2D(2,2),
•
•     layers.Conv2D(64, (3,3), activation='relu'),
•     layers.MaxPooling2D(2,2),
•
•     layers.Flatten(),
•     layers.Dense(128, activation='relu'),
•     layers.Dense(10, activation='softmax')
• ])
•
• # Compile model
• model.compile(
•     optimizer='adam',
•     loss='sparse_categorical_crossentropy',
•     metrics=['accuracy']
• )
```


-
- # Train model
- model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
-
- # Save model
- model.save("model/digit_model.h5")
-
- print("✔ Model trained and saved!")

2. App.py

```
import streamlit as st
import numpy as np
import cv2
from PIL import Image
from tensorflow.keras.models import load_model
import plotly.graph_objects as go
from streamlit_drawable_canvas import st_canvas

# -----
# LOAD MODEL
# -----
model = load_model("model/digit_model.h5")

# -----
# PAGE CONFIG
# -----
st.set_page_config(page_title="Digit Recognizer", layout="wide")

# -----
# CUSTOM CSS (BLUE + TRANSPARENT)
# -----
st.markdown("""
<style>
.stApp {
    background: linear-gradient(to right, #1e3c72, #2a5298);
    color: white;
}

/* Sidebar Blue */
section[data-testid="stSidebar"] {
    background: linear-gradient(to bottom, #1e3c72, #2a5298);
}

/* Remove boxes */
.block-container {
    background: transparent;

```

```

}

/* Buttons */
.stButton>button {
    background-color: #4B8BFF;
    color: white;
    border-radius: 10px;
}
</style>
""", unsafe_allow_html=True)

st.title("🔍 Handwritten Digit Recognition")

# -----
# HISTORY
# -----
if "history" not in st.session_state:
    st.session_state. History = []

# -----
# SIDEBAR (INSTRUCTIONS BACK)
# -----
st.sidebar.title("📌 Instructions")

st.sidebar.markdown("""
### How to Use:

1️⃣ Go to **Upload Tab** OR **Draw Tab**
2️⃣ Upload image OR draw digit
3️⃣ Click **Predict**
4️⃣ View result + graph

### Tips:
✓ Draw thick digit
✓ Keep center aligned
✓ Use white color

""")

# -----
# GRAPH FUNCTION (VERTICAL FIXED)
# -----
def show_graph(pred):

    digits = list(range(10))
    confidence = pred[0] * 100

    fig = go.Figure(data=[

```

```

        go.Bar(
            x=digits,
            y=confidence,
            width=0.6, # FIX: thickness
            marker=dict(
                color=[
                    '#FF4B4B', '#4B8BFF', '#00C49F', '#FFBB28', '#FF8042',
                    '#8E44AD', '#2ECC71', '#F39C12', '#E74C3C', '#1ABC9C'
                ]
            ),
            text=[f"{c:.1f}%" for c in confidence],
            textposition='outside'
        )
    ])

    fig.update_layout(
        title="Prediction Confidence",
        xaxis_title="Digits",
        yaxis_title="Confidence (%)",
        yaxis=dict(range=[0,100]),
        bargap=0.25, # FIX: spacing
        plot_bgcolor="rgba(0,0,0,0)",
        paper_bgcolor="rgba(0,0,0,0)",
        font=dict(color="white")
    )

    st.plotly_chart(fig, use_container_width=True)

# -----
# TABS
# -----
tab1, tab2, tab3 = st.Tabs(["📁 Upload", "🖌 Draw", "📜 History"])

# -----
# TAB 1 - UPLOAD
# -----
with tab1:

    uploaded_file = st.file_uploader("Upload Digit", type=["png",
    jpg", "jpeg"])

    if uploaded_file is not None:

        image = Image.open(uploaded_file).convert('L')
        st.image(image, width=200)

        if st.button("🔍 Predict Upload"):

```

```

        img = image.resize((28,28))
        img = np.array(img)

        img = 255 - img
        img = img / 255.0
        img = img.reshape(1,28,28,1)

        pred = model.predict(img)
        digit = np.argmax(pred)

        st.success(f"Prediction: {digit}")

        st.session_state.history.append(f"Upload → {digit}")

        show_graph(pred)

# -----
# TAB 2 - DRAW (FIXED CANVAS)
# -----
with tab2:

    st.subheader("Draw your digit below 🖊")

    # DEBUG TEXT (helps confirm rendering)
    st.write("Canvas loading...")

    canvas_result = st_canvas(
        fill_color="white",
        stroke_width=18,
        stroke_color="white",
        background_color="black",
        height=300,
        width=300,
        drawing_mode="freedraw",
        key="canvas_FIX_123", # IMPORTANT FIX (force rerender)
    )

    st.write("Canvas loaded")

    if st.button("🔍 Predict Drawing"):

        if canvas_result.image_data is not None:

            img = canvas_result.image_data[:, :, 0]
            img = cv2.resize(img, (28,28))

            img = img / 255.0
            img = img.reshape(1,28,28,1)

```

```

        pred = model.predict(img)
        digit = np.argmax(pred)

        st.success(f"Prediction: {digit}")

        st.session_state.history.append(f"Draw → {digit}")

        show_graph(pred)

    else:
        st.warning("Draw something first!")

# -----
# TAB 3 - HISTORY
# -----
with tab3:

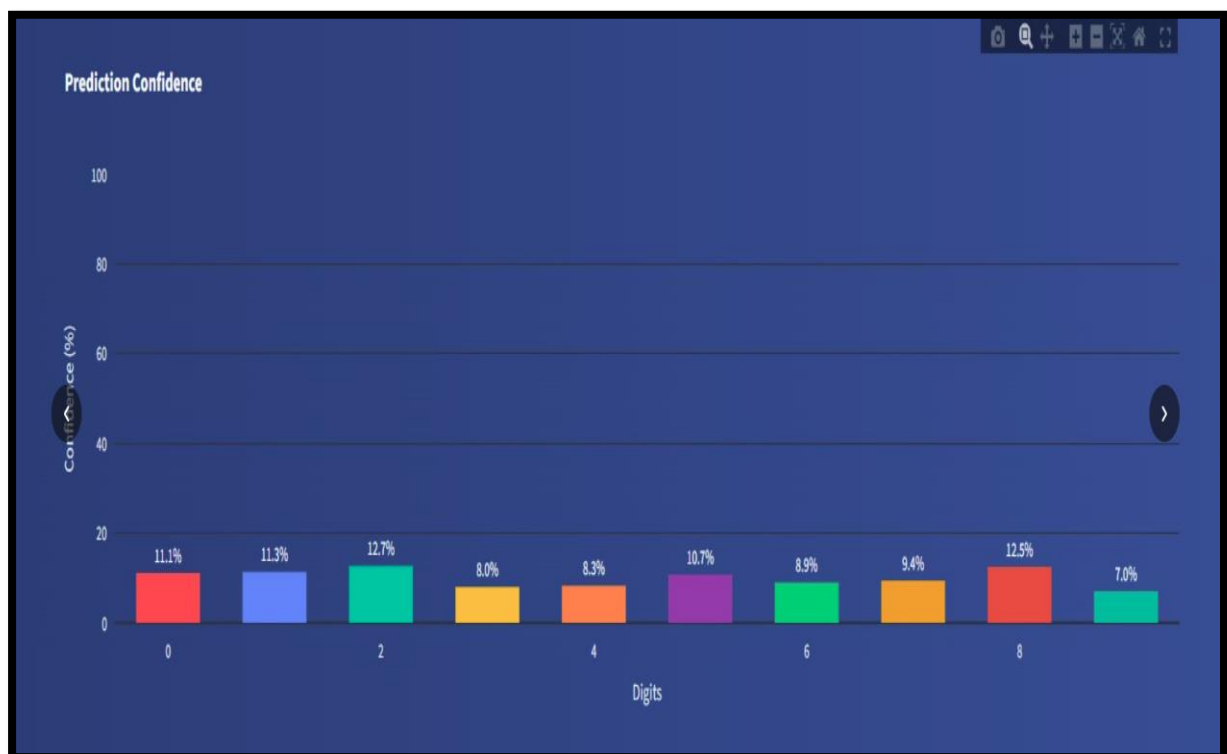
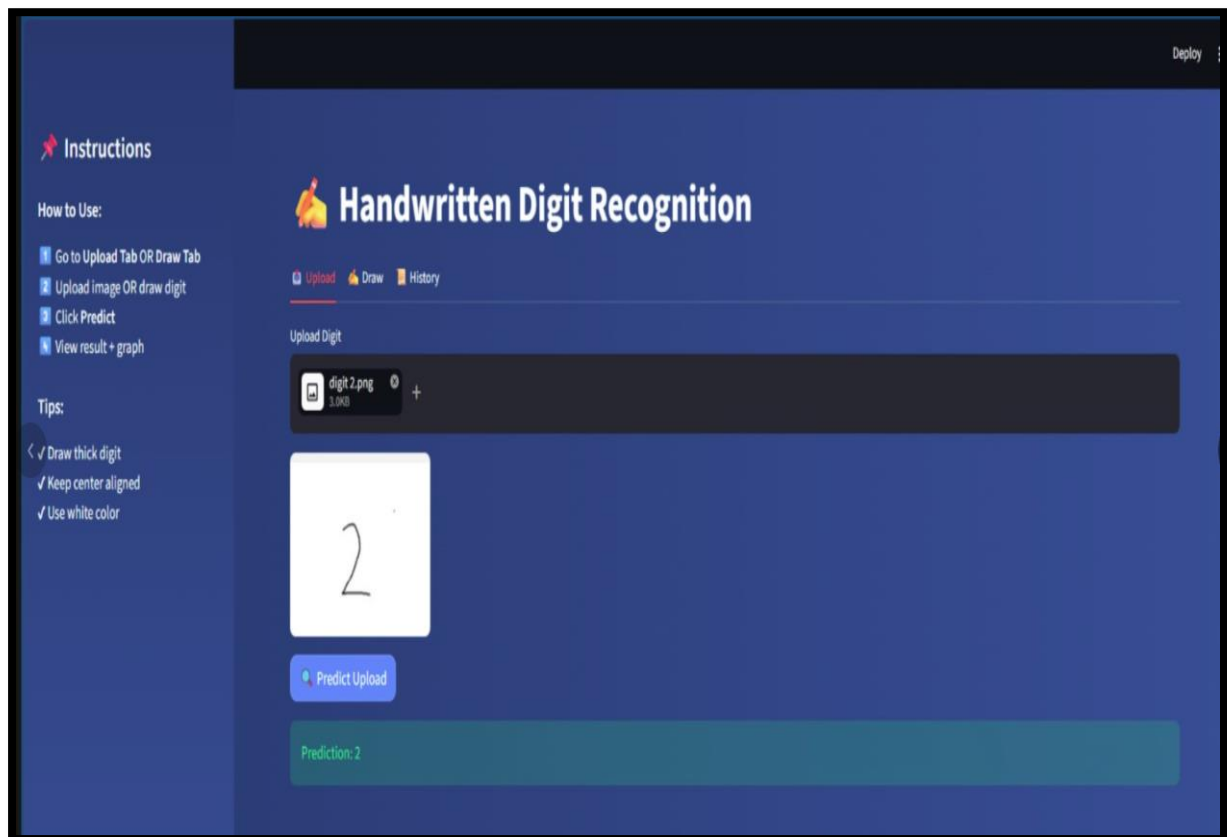
    st.subheader("📋 Prediction History")

    if len(st.session_state.history) == 0:
        st.info("No predictions yet")

    else:
        for i, item in enumerate(st.session_state.history, 1):
            st.write(f"{i}. {item}")

```

Appendix B Screenshot of Project





Handwritten Digit Recognition

Upload

Draw

History

Draw your digit below

Canvas loading...

Canvas loaded

Predict Drawing



Handwritten Digit Recognition

Upload

Draw

History



Prediction History

1. Upload → 2

Appendix C abbreviation

- **AI** – Artificial Intelligence
- **ML** – Machine Learning
- **DL** – Deep Learning
- **ANN** – Artificial Neural Network
- **CNN** – Convolutional Neural Network
- **MNIST** – Modified National Institute of Standards and Technology dataset
- **CSV** – Comma Separated Values
- **API** – Application Programming Interface
- **GUI** – Graphical User Interface
- **ReLU** – Rectified Linear Unit
- **Softmax** – Activation function used for multi-class classification
- **NumPy** – Numerical Python library
- **Pandas** – Python data analysis library
- **TensorFlow** – Open-source deep learning framework
- **Keras** – High-level neural network API
- **PIL** – Python Imaging Library
- **UI** – User Interface
- **HTML** – Hypertext Mark-up Language
- **CSS** – Cascading Style Sheets

References

1. Research papers related to image classification and machine learning techniques for pattern recognition
2. Python official documentation.
3. NumPy documentation for numerical feature extraction.
4. Pandas documentation for data analysis.
5. Scikit-learn documentation for machine learning utilities
6. MNIST dataset official source